

DataExpert.io Community Academy - Week 3 Spark Fundamentals Grader Feedback

Student

Name: Ceasar Jackson

Final Grade

B

Overall feedback

Hi Ceasar - thanks for a well-structured submission. You clearly put care into modularizing the work, and most requirements are implemented cleanly. The grader provided detailed feedback by requirement plus actionable improvements needed for full marks.

What you did well

Configuration and controls

- Disabled automatic broadcast joins with `spark.sql.autoBroadcastJoinThreshold = -1`.
- Used explicit broadcast () for the small medals and maps dimensions.
- AQE is enabled and does not negate the intentional broadcast-control strategy.

Bucketing

The submission writes `match_details`, `matches`, and `medals_matches_players` as 16-bucket Hive tables on `match_id`, using `saveAsTable`, and re-reads them for joins. This retains catalog bucketing metadata and is the appropriate approach for bucket-aware joins.

Join strategy

`build_joined_dataframe` broadcasts medals and maps and joins the three bucketed fact-like datasets. Aliases are used appropriately, and medal joins are deferred until after establishing match/player grain to reduce row-explosion risk.

Aggregations

- **4a:** First aggregates to (`player_gamertag`, `match_id`) to protect average kills from duplication by downstream joins. Correct approach.
- **4b:** Most-played playlist using distinct matches is correct and efficient.
- **4c:** Most-played map uses an explicit broadcast join and handles map-name fallback gracefully.
- **4d:** Killing Spree filtering is case-insensitive and sums `mmp.count` per map correctly.

Storage and size experiments

Multiple `sortWithinPartitions` orders were tested over a compacted aggregate keyed by low-cardinality fields, and the script reports a winning layout. The narrative and size-measurement helper demonstrate good engineering practice.

Gaps and recommended fixes

Requirement 3 - Ensure all three bucketed tables are actually bucket-joined

A bucketed join is likely achieved between `match_details` and `matches` on `match_id`.

The join to `medals_matches_players`, however, uses two predicates:

```
md.match_id == mmp.match_id
AND
md.player_gamertag == mmp.player_gamertag
```

Because Spark bucket-aware joins generally require the join keys to align with the bucketing keys, the additional `player_gamertag` predicate can prevent that leg from being recognized as a bucketed join on `match_id`.

Recommended correction

1. Join `match_details` to `matches` on `match_id`.
2. Join that result to `medals_matches_players` on `match_id` so the bucket key aligns.
3. Handle player-level filtering or aggregation afterward as needed to prevent duplication.
4. Capture `joined.explain("formatted")` output as evidence.
5. Identify the bucket-aware join in the physical plan and explain any leg that cannot use bucket-aware execution because of additional join keys.

Requirement 5 - Partitioned tables plus `sortWithinPartitions`

This is the major completeness gap identified by the grader.

The original experiment varies intra-partition sorting, but it does not sufficiently vary physical partitioning with `repartition` / `partitionBy`, nor write at least three versions of partitioned tables.

The assignment explicitly asks for multiple partitioned-table versions using `sortWithinPartitions`, focused on low-cardinality fields.

Recommended experiment variants

```
aggregated.repartition("playlist_id").sortWithinPartitions("playlist_id", "map_name")  
.write.mode("overwrite").partitionBy("playlist_id").parquet(path)
```

```
aggregated.repartition("map_name").sortWithinPartitions("map_name", "playlist_id").write.mode("overwrite")  
.partitionBy("map_name").parquet(path)
```

```
aggregated.coalesce(k).sortWithinPartitions("playlist_id").write.mode("overwrite").parquet(path)
```

Useful comparisons include:

- `partitionBy("playlist_id")` with and without intra-partition sorting.
- `partitionBy("map_name")` with and without intra-partition sorting.
- Controlled non-partitioned baseline.
- Explicit `repartition` choices to make comparisons reproducible and fair.

Storage-size measurement

If output is stored on DBFS, S3, or HDFS, a local `directory_size_bytes()` helper may not report correct values. Use the platform filesystem interface instead, such as `recursive dbutils.fs.ls`, Hadoop `FileSystem` APIs, or S3 object listings, and persist the measured results.

Minor correctness and robustness points

Qualified column references

Using a reference such as `mmp["medal_id"]` after `mmp` has already become part of a joined left-hand `DataFrame` can depend on qualifier preservation.

Safer approaches include:

```
F.col("mmp.medal_id")
```

or explicitly selecting/renaming the needed columns after the first join.

Map-name grouping reference

Inside a joined DataFrame, prefer:

```
F.col(map_name_col)
```

or, with an alias:

```
F.col(f"map.{map_name_col}")
```

rather than referencing the original maps DataFrame object.

Schema validation

The code already handles alternate medal/map naming conventions. The grader recommends extending required-column checks to include:

- match_details: player_gamertag, player_total_kills
- matches: playlist_id, map_id
- medals_matches_players: medal_id, count
- maps: map_id, and optionally a name field

Spark configuration

Consider explicitly setting `spark.sql.shuffle.partitions` to an appropriate value for the execution environment and documenting that setting for reproducibility.

Evidence and reproducibility

The grader recommends persisting evidence rather than only printing it to the console.

Save:

1. Formatted physical plans to the output directory.
2. Evidence demonstrating bucket-aware and broadcast join behavior.
3. The size-comparison DataFrame as Parquet.
4. A small CSV or JSON summary of the storage experiment.

5. Relevant schemas and sample records when environment assumptions need verification.

Additional information requested if environment differs

If the prompt or schema differs from the grader's assumptions, provide `printSchema()` and `show(5, truncate=False)` output for:

- `match_details`
- `matches`
- `medals_matches_players`
- `medals`
- `maps`

Also provide the formatted physical plan for the primary joined DataFrame and identify the storage layer if using DBFS, S3, or HDFS.

Final grader assessment

Correctness

Strong analytical queries and explicit broadcast usage, with a minor risk around qualified column references.

Performance and optimizations

Good use of bucketing and broadcast hints. The submission **partially meets** the "partitioned tables + `sortWithinPartitions`" requirement; adding genuine `partitionBy` / `repartition` experiments is necessary to complete it.

Code quality

Clear structure, useful comments, early validation, and a sound aggregation strategy for avoiding duplication.

Final Grade: B

Remediation priorities

Based on the grader feedback, the highest-priority work for an A-level resubmission is:

1. Prove bucket-aware join behavior with persisted formatted physical-plan evidence.
2. Correct or restructure the `medals_matches_players` join if needed so bucket keys align.

3. Add at least three genuine physical partitioning experiments using `partitionBy` / `repartition` plus `sortWithinPartitions`.
4. Measure output sizes using the correct filesystem API.
5. Persist experimental results and explain plans as reproducibility evidence.
6. Harden qualified-column references and schema validation.